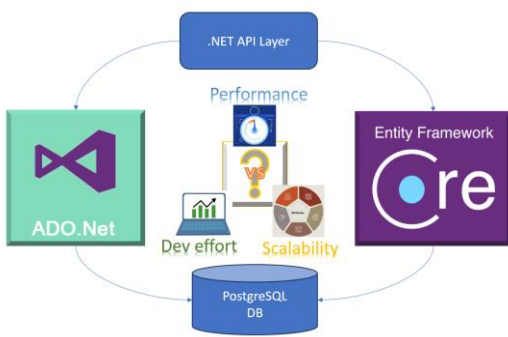




Department of Software Engineering

Project team: ID:123 KSI	1. Valentyna Yakushenko - leader 2. Qinze Xue 3. Najat Mohammed
Supervisor:	dr inż. Bartosz Czaplewski
Client:	Marlena Dzierma (Ironpack Sp. z o.o. Sp.k.)
Date:	13.06.2025
Key words:	Ticket system, enterprise web application, Entity Framework Core, ADO.NET, comparative analysis, database access, performance, maintainability



```
graph TD; API[.NET API Layer] --> ADO[ADO.Net]; API --> EF[Entity Framework Core]; ADO --> DB[(PostgreSQL DB)]; EF --> DB; ADO -.-> P[Performance]; ADO -.-> DE[Dev effort]; ADO -.-> S[Scalability]; EF -.-> P; EF -.-> DE; EF -.-> S; DB -.-> P; DB -.-> DE; DB -.-> S;
```

The diagram illustrates a layered architecture for a ticketing system. At the top is the **.NET API Layer**, which connects to both **ADO.Net** and **Entity Framework Core**. Both of these connect to the **PostgreSQL DB** at the bottom. The diagram also highlights key metrics: **Performance**, **Dev effort**, and **Scalability**, which are influenced by the database layer and the application logic.

PROJECT TITLE:

Comparative Analysis of ORM and Direct SQL Data Access in an Internal Ticketing System

RESEARCH THESIS, OBJECTIVES AND SCOPE:

The goal of this project is to design and develop a web-based internal Ticketing System that enables employees of *Ironpack Sp. z o.o. Sp.k.* to report technical, logistical, or administrative issues, and allows support teams to manage and resolve them efficiently. The system is designed to follow a layered architecture using modern technologies (SvelteKit, .NET 8, PostgreSQL) to ensure usability, scalability, and maintainability.

In addition to building a real-world internal ticketing application for *Ironpack Sp. z o.o. Sp.k.* this project investigates the comparative advantages of two commonly used approaches for database access in .NET-based applications:

- Object-Relational Mapping (ORM) using Entity Framework Core (EF Core)
- Direct SQL access using ADO.NET with raw SQL queries

The objective of this research is to evaluate and compare development effort (code complexity, maintainability) and runtime performance (execution speed) for both approaches, using realistic ticket operations (inserts and queries). As a result, the research will provide practical recommendations for the company and academic community on selecting the most suitable data access strategy for similar enterprise web applications.

The scope includes implementing core ticket operations using both methods, measuring development time, code size and basic performance metrics; reporting findings in a clear, actionable format.

RESULTS:

1. Functional and non-functional requirement analysis for the ticketing system, based on company needs and user expectations
2. Scope definition, assumptions and identification of the main user groups and ticket categories
3. System concept development – ticket workflow diagram, database model, and preliminary architecture
4. Selection and justification of implementation technologies (SvelteKit, .NET 8, PostgreSQL, Entity Framework Core/ADO.NET)
5. Research subject definition and determination of scalability and performance metrics



MAIN FEATURES, FUTURE WORKS:

Main Features:

- Ticket Management: Submission, tracking, assignment and resolution of technical, logistics and administrative issues.
- Role-Based Access Control: Customized permissions for Employees, Support Staff, Team Leaders, and Administrators.
- Dual-language support: Polish/English interface and notification
- Notifications: Email updates for ticket status changes
- Analytics Dashboard: Charts, statistics and reports for administrators and team leaders
- Ticket History: Logging of all changes, comments, and assignments for full transparency
- Advanced search and filtering for all users
- User-friendly and responsive web application

Future works:

- Implementation of real-world web application with two defined approaches for database access
- Comparative Analysis of database access methods focusing on
 - Development effort (implementation time, code complexity, maintainability)
 - System performance (query and operation execution time)
 - Scalability (stability and responsiveness under simulated high load)